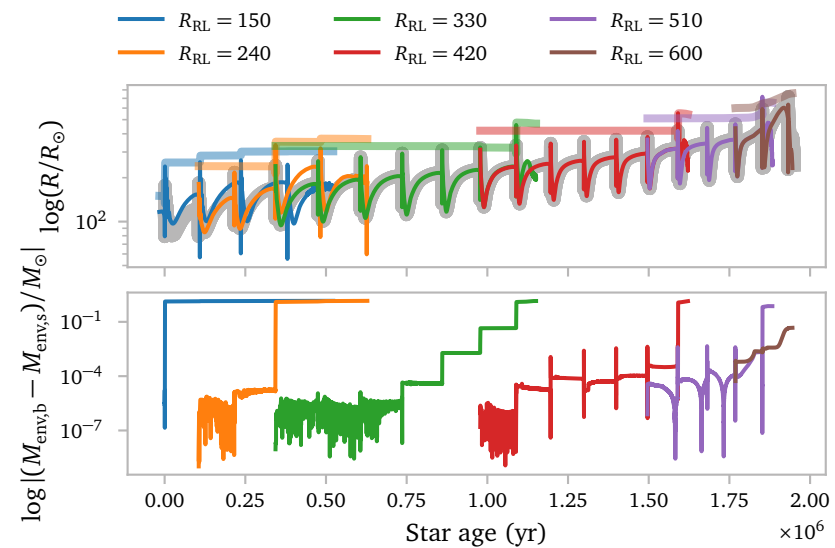
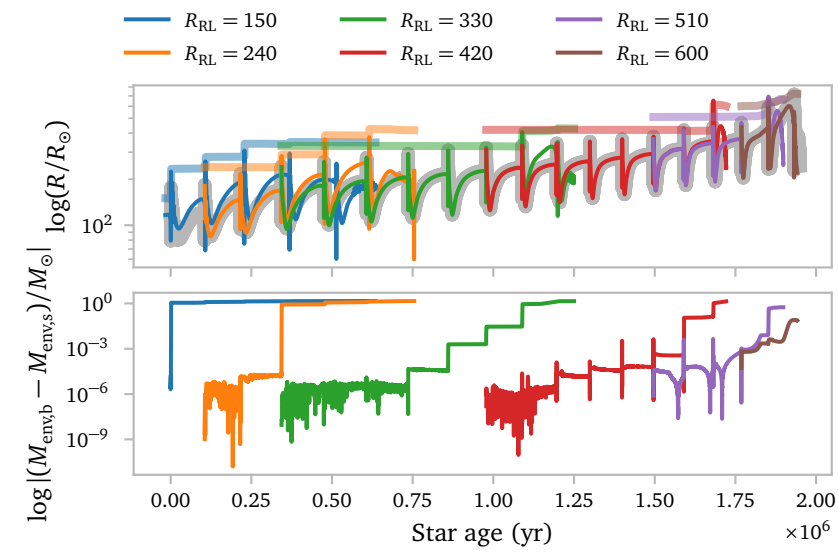
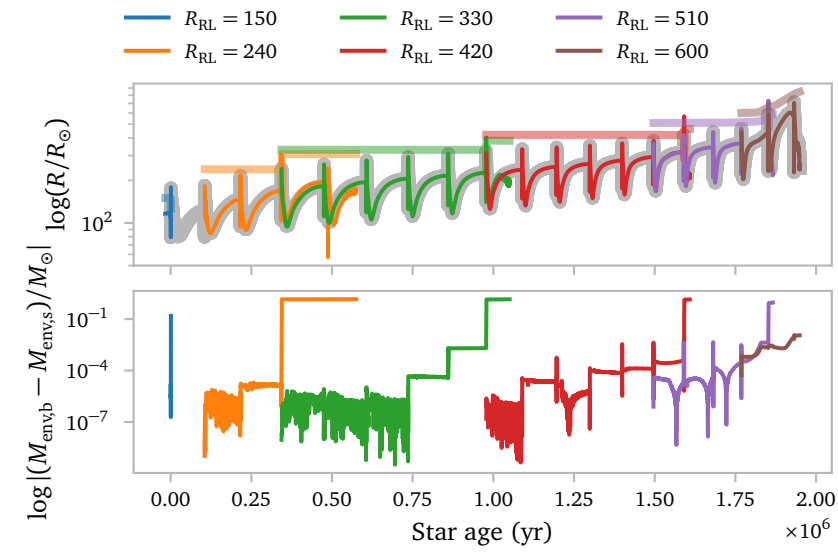
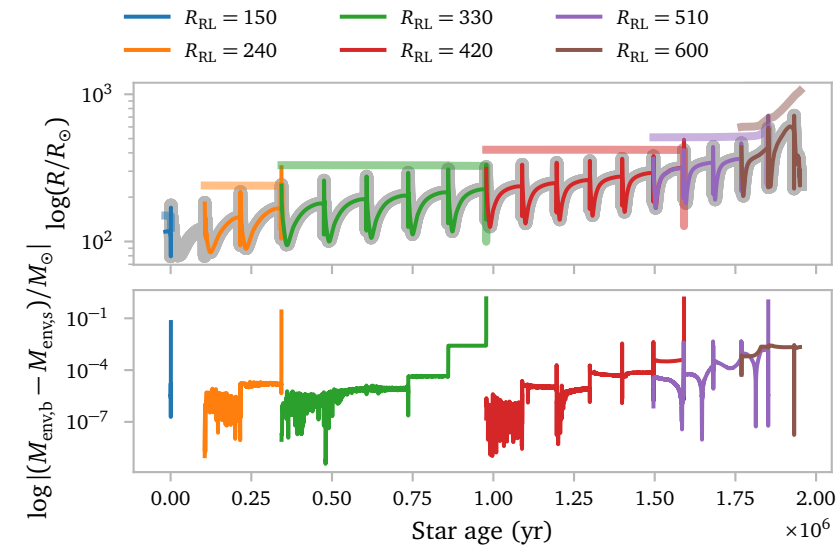


Notes Week 8

a. bla



b. Additional plots



c. Tidal Synchronization

c.1 Theory

Here, I describe the (I think) relevant results from [Hut \(1981\)](#) and [Preece et al. \(2022\)](#). In particular, I describe the tidal evolution equations (9), (10) and (11) from [Hut \(1981\)](#):

$$\begin{aligned}\frac{da}{dt} &= -6 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 \frac{a}{(1-e^2)^{15/2}} \left[f_1(e^2) - (1-e^2)^{3/2} f_2(e^2) \frac{\Omega}{n} \right], \\ \frac{de}{dt} &= -27 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 \frac{e}{(1-e^2)^{13/2}} \left[f_3(e^2) - \frac{11}{18} (1-e^2)^{3/2} f_4(e^2) \frac{\Omega}{n} \right], \\ \frac{d\Omega}{dt} &= 3 \frac{k}{T} \frac{q^2}{r_g^2} \left(\frac{R}{a}\right)^6 \frac{n}{(1-e^2)^6} \left[f_2(e^2) - (1-e^2)^{3/2} f_5(e^2) \frac{\Omega}{n} \right].\end{aligned}$$

Here, f_i is a family of functions only dependent on e^2 which a nice similarity, namely that when $e = 0$, $f_i = 1$. Since we are (at least now,) only looking at circular orbits ($e = 0$), the equations above simplify to

$$\frac{da}{dt} = -6 \frac{k}{T} q(1+q) \left(\frac{R}{a}\right)^8 a \left[1 - \frac{\Omega}{n} \right], \quad (1)$$

$$\frac{de}{dt} = 0, \quad (2)$$

$$\frac{d\Omega}{dt} = 3 \frac{k}{T} \frac{q^2}{r_g^2} \left(\frac{R}{a}\right)^6 n \left[1 - \frac{\Omega}{n} \right]. \quad (3)$$

In these equations, n^1 is the mean orbital angular velocity and r_g is the radius of gyration².

A last important factor in these equations is the ratio k/T , which contains the dependence of the equations on the structure of the star ([Preece et al., 2022](#)). Here, the k is the *apsidal motion* constant³ and T is the tidal response time.

This ratio can be approximated following [Preece et al. \(2022\)](#) as

$$\left(\frac{k}{T}\right)_{\text{conv}} = \left(\frac{R_{\text{conv}}}{R}\right)^a \left(\frac{M_{\text{conv}}}{M}\right)^b \frac{c}{t_{\text{conv}}}, \quad (4)$$

where a , b and c are functions of the metallicity z only, and t_{conv} is given by

$$t_{\text{conv}} = 0.4311 \left[\frac{3M_{\text{conv}} R_{\text{conv}}^2}{L} \right]^{1/3} \text{ yr}. \quad (5)$$

For completeness, I also list the equations for a , b and c below as well:

$$a \approx 2.72 + 0.63 \cdot \log z,$$

$$b \approx 0.68 - 0.219 \cdot \log z,$$

$$c \approx 0.12 - 0.023 \cdot \log z.$$

Lastly, a correction factor f_{conv} for *fast tides*⁴ can be used to change k/T in the fast tide regime.

There exist multiple prescriptions for choosing sensible values for f_{conv} . I only describe the prescription by [Goldreich & Nicholson \(1989\)](#) as it is implemented in *Binary Star Evolution* (BSE) ([Hurley et al., 2002](#)):

$$f_{\text{conv}} = \min \left[1, \left(\frac{P_{\text{tid}}}{2t_{\text{conv}}} \right)^2 \right],$$

where P_{tid} is the tidal pumping timescale given by

$$\frac{1}{P_{\text{tid}}} = \left| \frac{1}{P_{\text{orb}}} - \frac{1}{P_{\text{spin}}} \right|.$$

In the case of a radiative envelope, the ratio k/T should be damped, but since TPAGB stars only have convective envelopes, there is (at least at this point) no need to describe and implement this damping.

¹ $n \equiv G^{1/2}(M+m)^{1/2}a^{-3/2}$

² r_g is defined as $I = M(r_g R)^2$, where I the moment of inertia of the primary.

³ Does this relate to how much an elliptical orbit will rotate (apsidal precession)?

⁴ Regime where the tidal period (what is the tidal period?) is shorter than the convective turnover time t_{conv} .

c.2 MESA Toy Model

MESA has an implementation for tides in binary stars. This option has to be turned on in the binary inlist settings using `do_tidal_sync`. The implementation of the tidal interactions is interiorly written in the file `$MESA/binary/private/binary_tides.f90` and largely follows the theory described above.

There are, however, small differences. To start, MESA does not use the fit for k/T from [Preece et al. \(2022\)](#), but a similar fit from [Hurley et al. \(2002\)](#). Then, MESA actually only directly computes the change of the spin angular frequency of the star(s). By keeping track of the lost or gained angular momentum, the orbital separation and eccentricity are then adjusted using conservation of angular momentum elsewhere. Below, I show the piece of code that is responsible for adjusting the spin angular momentum in the default implementation of tides in MESA:

```
! binary_tides.f90
subroutine sync_spin_to_orbit(...)
...
  if (sync_mode == "Uniform") then
    a1 = f2(b% eccentricity)
    a2 = pow(1-pow2(b% eccentricity), 1.5d0)*f5(b% eccentricity)
    do k=1,nz
      delta_j(k) = (1d0 - exp(-a2*dt_next/t_sync))*(s% j_rot(k) - a1/a2*j_sync(k))
    end do
...

  if (.not. b% doing_first_model_of_run) then
    do k=1,nz
      s% extra_jdot(k) = s% extra_jdot(k) - delta_j(k)/dt_next
    end do
  end if

end subroutine sync_spin_to_orbit
```

The `s% extra_jdot(k)` is used to spin the star up or down. In most cases, this is exactly what we want, but in our case, where we want to only see the effects of the tides, and preferably keep a naive non-rotating star, this is not what we want.

To only affect the *orbit*, we need to bypass the star interiorly and update the orbital angular momentum directly. I think the easiest way to do is to utilize the `other_jdot_ls` hook. By enabling this other hook in the binary inlist, and writing the following line in the `extras_run_binary_controls`:

```
b% other_jdot_ls => toy_tides
```

we can now implement our own tide formulation. As the name of the `other_jdot_ls` hook reveals, my first try will just be a very simple toy example. Below, I write down my code:

```
! src/bin/additional_routines.inc
subroutine toy_tides(binary_id, ierr)
! --- following the required input and output of default_jdot_ls
integer, intent(in) :: binary_id
integer, intent(out) :: ierr

! --- getting access to the binary object and star object
type(binary_info), pointer :: b
type(star_info), pointer :: s

! --- necessary other variables
real(dp) :: omega, omega_orb, domega_dt
real(dp) :: I_eff, Jdot_tides, dt
real(dp) :: t_sync
integer :: k ! for counting zones

ierr = 0
call binary_ptr(binary_id, b, ierr)
s => b% s_donor

dt = s% dt

! --- get initial spin and orbital frequency
omega = s% x_ctrl(3)
omega_orb = 2d0*pi / b% period

! --- arbitrary sync timescale
t_sync = 1d12 ! in seconds (30000 yr)

! --- simplified hut spin evolution
domega_dt = (omega_orb - omega) / t_sync

! --- update spin
omega = omega + dt * domega_dt
s% x_ctrl(3) = omega

! --- compute moment of inertia of solid body
I_eff = 0
do k=1,s% nz
  I_eff = I_eff + s% dm_bar(k) * (s% r(k)**2)
end do
Jdot_tides = I_eff * domega_dt

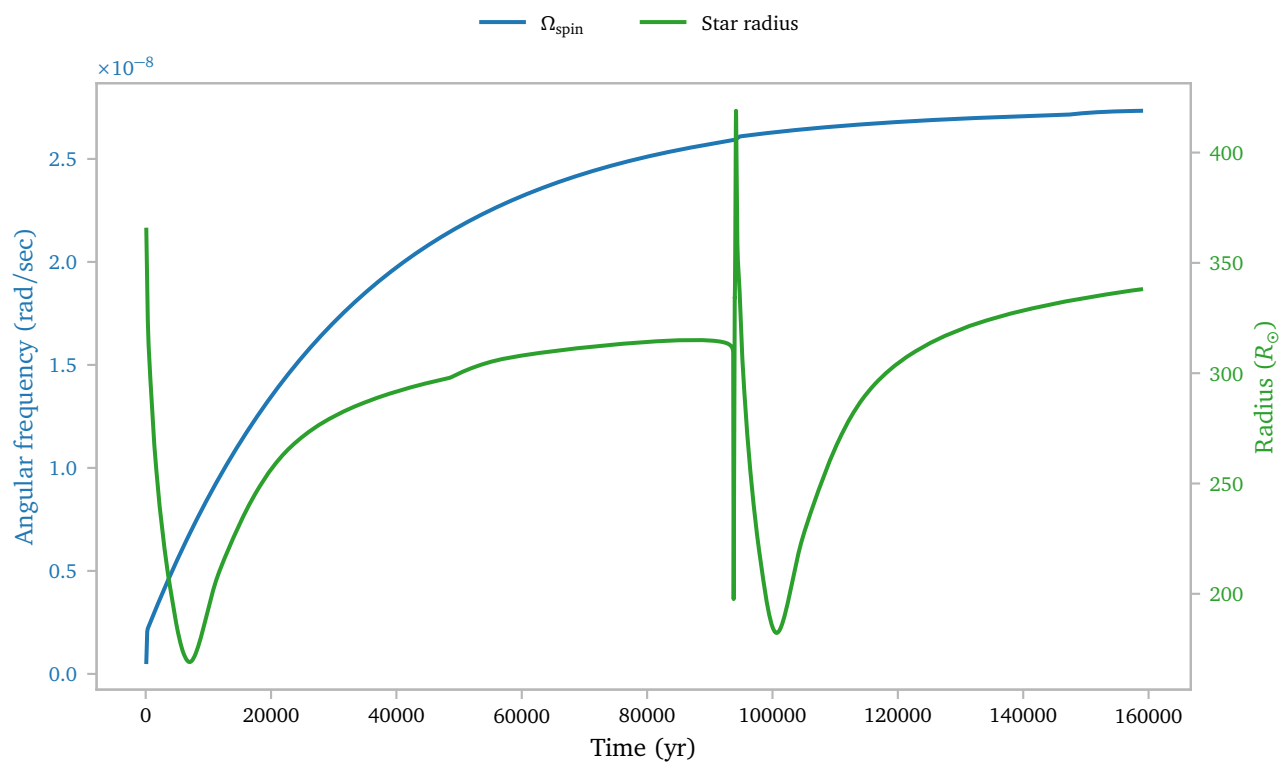
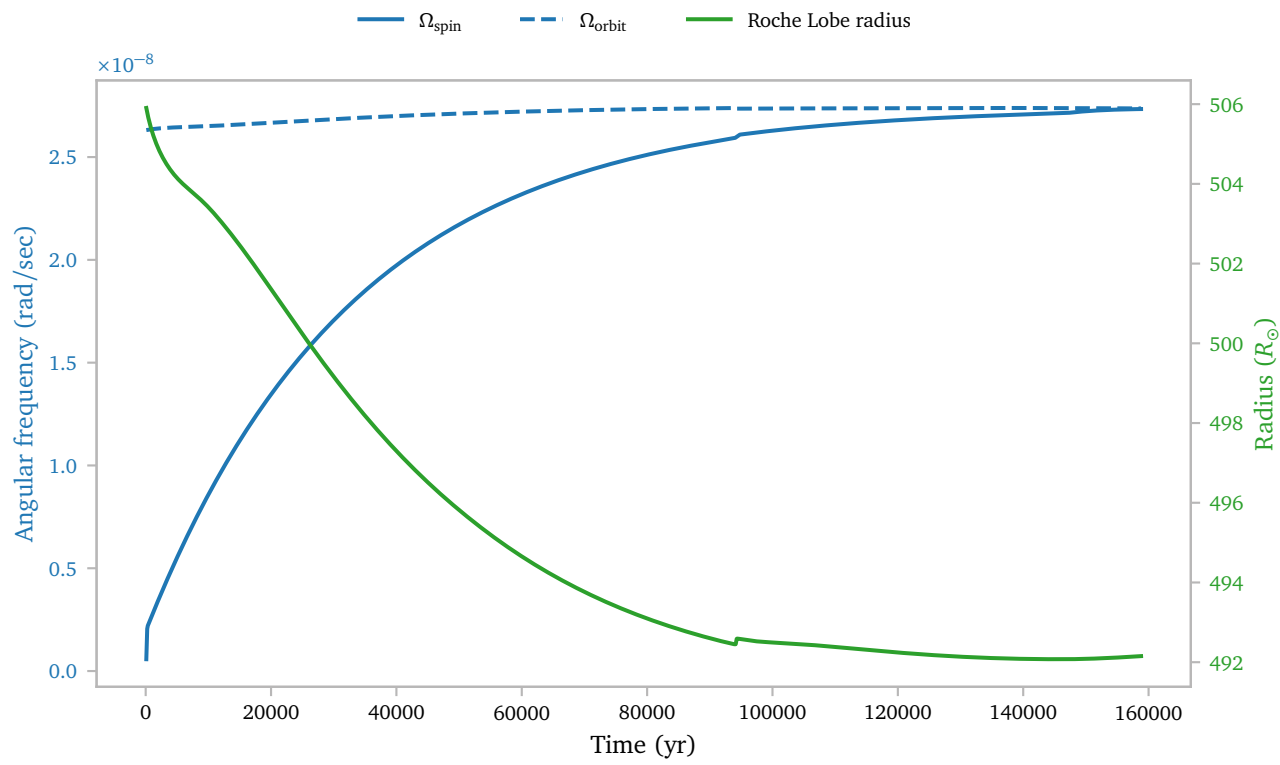
! --- change jdot orbit
b% jdot_ls = - Jdot_tides

! --- write orbit and spin frequency
write(*,*) "Omega_star = ", omega
write(*,*) "Omega_orb = ", omega_orb
write(*,*) "J_orb =", b% angular_momentum_j
write(*,*) "a =", b% separation

end subroutine toy_tides
```

I also add the spin angular frequency and the orbit angular frequency to the history columns to visualize the effect of the tides.

It is important to realize that the toy model is an extremely simplified version of a full tidal implementation. Below, I plot the results of the simple model, first for a initially non-rotating star.



It is obvious that the simple tidal model works in the sense that the difference between the orbital angular frequency and spin angular frequency approach eachother. Since the star starts out with a slower spin, the orbit decreases (and thus the Roche lobe radius too). The second plot however, shows that the spin of the star at this point, is completely unaffected by changes in the structure of the star. During a radius peak, conservation of angular momentum should cause the star to show down its rotation, but this does not happen.

I think this is nice and easy improvement to make to the toy model:

c.3 MESA Toy Model with Stellar Evolution

```
! src/bin/additional_routines.inc
subroutine toy_tides_with_star_evo(binary_id, ierr)
  ! --- following the required input and output of default_jdot_ls
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr

  ! --- getting access to the binary object and star object
  type(binary_info), pointer :: b
  type(star_info), pointer :: s

  ! --- necessary other variables
  real(dp) :: omega, omega_orb, domega_dt
  real(dp) :: I_eff, Jdot_tides, dt
  real(dp) :: I_old, dlnI_dt, t_sync
  real(dp) :: domega_dt_tides

  integer :: k ! for counting zones

  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  s => b% s_donor

  dt = s% dt

  ! --- get initial spin and orbital frequency
  omega = s% x_ctrl(3)
  omega_orb = 2d0*pi / b% period

  ! --- arbitrary sync timescale
  t_sync = 1d12

  ! --- compute moment of inertia of solid body
  I_eff = 0
  do k=1,s% nz
    I_eff = I_eff + s% dm_bar(k) * (s% r(k)**2)
  end do

  ! --- compute dlnI/dt
  I_old = s% x_ctrl(4)
  if (I_old > 0d0 .and. dt > 0d0) then
    dlnI_dt = (log(I_eff) - log(I_old)) / dt
  else
    dlnI_dt = 0d0
  end if

  ! --- simplified hut spin evolution
  domega_dt_tides = (omega_orb - omega) / t_sync

  ! --- total spin evolution (structure + tides)
  domega_dt = domega_dt_tides - omega * dlnI_dt

  ! --- update spin
  omega = omega + dt * domega_dt
  s% x_ctrl(3) = omega

  ! --- store I for next step
  s% x_ctrl(4) = I_eff

  ! --- change jdot orbit
  Jdot_tides = I_eff * domega_dt_tides
  b% jdot_ls = - Jdot_tides

  ! --- write orbit and spin frequency
  write(*,*) "Omega_star = ", omega
  write(*,*) "Omega_orb = ", omega_orb
  write(*,*) "J_orb =", b% angular_momentum_j
  write(*,*) "a =", b% separation

end subroutine toy_tides_with_star_evo
```

References

Goldreich, P. & Nicholson, P. D. (1989) *Tidal Friction in Early-Type Stars*. [ApJ342, 1079](#).

Hurley, J. R., Tout, C. A., & Pols, O. R. (2002) *Evolution of binary stars and the effect of tides on binary populations*. [MNRAS329, 897](#).

Hut, P. (1981) *Tidal evolution in close binary systems*. [A&A99, 126](#).

Preece, H. P., Hamers, A. S., Neunteufel, P. G., Schaefer, A. L., & Tout, C. A. (2022) *The Equilibrium Tide: An Updated Prescription for Population Synthesis Codes*. [ApJ933, 25](#).